Практична робота. Python: модульність, функції, бібліотеки та модулі (11 клас)

Що робимо

Ви напишете чотири програми на Python, у яких розіб'єте задачу на функції, скористаєтеся готовими модулями `math` і `random` та створите власний модуль. Код пишемо в онлайн-редакторі OnlineGDB: нічого встановлювати й реєструватися не потрібно, модулі `math` і `random` уже вбудовані, а для власного модуля можна додати в проєкт другий файл. Запасні варіанти — у розділі «Де писати код і як поділитися».

1. Теорія

Модульність

Модульність — це принцип, за яким велику програму складають із невеликих незалежних частин. Кожна частина розв'язує одну задачу, її можна окремо написати, перевірити й використати повторно. У Python такими частинами є **функції** (частина програми) і **модулі** (окремий файл із функціями).

Функція

Функція — це іменований фрагмент коду, який виконується, коли його викликають за іменем. Ви вже користувалися вбудованими функціями: `print()`, `input()`, `int()`, `len()`. Тепер створимо власні.

```
def greet(name):
    print("Привіт,", name)

greet("Олена")
greet("Тарас")
```

- `def` — ключове слово, з якого починається опис функції.
- `greet` — ім'я функції (правила ті ж, що й для змінних).
- `name` — **параметр**: змінна, яка отримує значення під час виклику.
- `"Олена"` — **аргумент**: конкретне значення, передане у функцію.

Рядок з `def` завершує двокрапка, тіло функції записують із відступом у 4 пробіли. Опис функції сам нічого не робить: код усередині виконується лише під час виклику `greet(...)`. Тому функцію треба описати **вище** від місця, де її викликають.

Повернення результату: return

`return` завершує функцію й повертає значення в те місце, звідки її викликали. Результат можна записати в змінну або одразу використати у виразі.

```
def square(x):
    return x * x

a = square(5)
print(a)                    # 25
print(square(3) + square(4))  # 25
```

Функція без `return` повертає спеціальне значення `None`. Тому зазвичай функція **обчислює й повертає** результат, а виводить його на екран основна програма.

Кілька параметрів і значення за замовчуванням

```
def rect_area(a, b=1):
    return a * b

print(rect_area(4, 5))  # 20
print(rect_area(4))     # 4, бо b = 1 за замовчуванням
```

Функції з розгалуженням

Усередині функції можна використовувати все, що ви вже знаєте, зокрема `if-elif-else`. Щойно виконується `return`, функція завершується.

```
def sign(n):
    if n > 0:
        return "додатне"
    elif n < 0:
        return "від'ємне"
    else:
        return "нуль"

print(sign(int(input("Число: "))))
```

Локальні змінні

Змінні, створені всередині функції (і її параметри), є **локальними**: вони існують лише під час виконання функції. Поза функцією до них звернутися не можна.

```
def double(x):
    result = x * 2
    return result

print(double(7))  # 14
print(result)     # NameError: name 'result' is not defined
```

Модулі та бібліотеки

Модуль — це файл `.py` з функціями та змінними, які можна підключити до іншої програми. **Бібліотека** — набір модулів. Разом із Python встановлюється **стандартна бібліотека**: сотні готових модулів, які не треба завантажувати окремо.

Модуль	Для чого	Приклад
`math`	математичні функції й константи	`math.sqrt(16)` → `4.0`, `math.pi` → `3.14159…`
`random`	випадкові числа	`random.randint(1, 6)` → ціле від 1 до 6
`datetime`	дата й час	`datetime.date.today()` → сьогоднішня дата
`statistics`	статистика	`statistics.mean([2, 4, 6])` → `4`

Щоб скористатися модулем, його підключають командою `import` на початку програми. Є три способи:

```
import math
print(math.sqrt(25))      # звертаємося через ім'я модуля

from random import randint
print(randint(1, 10))     # функцію імпортовано, пишемо без префікса

import statistics as st
print(st.mean([3, 5, 7])) # скорочене ім'я модуля
```

Крім стандартної, є тисячі **сторонніх бібліотек** (наприклад, `numpy` для обчислень, `matplotlib` для графіків). Їх встановлюють окремо командою `pip install`, у цій роботі вони не знадобляться.

Власний модуль

Будь-який файл `.py` можна підключити як модуль. Створіть у проєкті другий файл, наприклад `mymath.py`, і запишіть у нього функції:

```
# файл mymath.py
def cube(x):
    return x ** 3

def average(a, b):
    return (a + b) / 2
```

У головному файлі `main.py` підключіть модуль за ім'ям файлу **без** `.py`:

```
# файл main.py
import mymath

print(mymath.cube(3))        # 27
print(mymath.average(4, 7))  # 5.5
```

Обидва файли мають лежати в одній папці (в одному проєкті онлайн-сервісу). Ім'я модуля не повинно збігатися зі стандартним: файл `math.py` або `random.py` «заступить» справжній модуль.

Типові помилки

- Забули дужки під час виклику: `square` замість `square(5)` — функція не виконується.
- Викликали функцію вище, ніж описали: `NameError: name 'square' is not defined`.
- Використали `print` замість `return`: `a = square(5)` дає `None`.
- Кількість аргументів не відповідає параметрам: `rect_area()` дає `TypeError: missing 1 required positional argument`.
- Забули імпорт: `sqrt(16)` без `from math import sqrt` дає `NameError`.
- Написали `import mymath.py`: `ModuleNotFoundError`, ім'я модуля пишуть без розширення.

2. Практична робота

Мета: навчитися описувати й викликати власні функції з параметрами та `return`, підключати модулі стандартної бібліотеки (`math`, `random`) і створювати власний модуль.

Кожне завдання робіть окремою програмою (завдання 4 — окремим проєктом із двох файлів). У кожній програмі спочатку опишіть функції, а нижче запишіть основну програму, яка вводить дані, викликає функції й виводить результат. Функції мають **повертати** результат через `return`, а не виводити його самі.

№	Завдання	Приклад
1	**Конвертер температур.** Опишіть дві функції: `c_to_f(c)` переводить градуси Цельсія у Фаренгейта за формулою `f = c * 9 / 5 + 32`, `f_to_c(f)` — навпаки, `c = (f - 32) * 5 / 9`. Основна програма питає, у який бік переводити (`1` або `2`), вводить температуру й виводить результат, округлений до 1 знака (`round(x, 1)`)	`1`, `100` → `212.0`; `2`, `98.6` → `37.0`
2	**Геометрія з math.** Підключіть модуль `math`. Опишіть функції: `circle_area(r)` — площа круга `π·r²`, `circle_length(r)` — довжина кола `2·π·r`, `hypotenuse(a, b)` — гіпотенуза прямокутного трикутника `√(a² + b²)`. Використайте `math.pi` і `math.sqrt`. Програма вводить радіус і катети та виводить усі три результати з точністю до 2 знаків	`r = 3` → площа `28.27`, довжина `18.85`; `a = 3`, `b = 4` → `5.0`
3	**Гра в кості.** Підключіть `randint` з модуля `random`. Опишіть функцію `roll()`, яка повертає випадкове число від 1 до 6, і функцію `winner(a, b)`, яка повертає `"Гравець"`, `"Комп'ютер"` або `"Нічия"`. Програма «кидає» кубик для гравця й комп′ютера, виводить обидва числа й переможця	`Гравець: 5`, `Комп'ютер: 2` → `Переміг Гравець`

№	Завдання	Приклад
4	**Власний модуль.** Створіть проєкт із двох файлів. У файл `geometry.py` перенесіть функції із завдання 2 і додайте `rect_area(a, b)` та `rect_perimeter(a, b)`. У файлі `main.py` підключіть модуль командою `import geometry` і виведіть площу й периметр прямокутника, площу круга та гіпотенузу для введених даних	`a = 4`, `b = 6` → площа `24`, периметр `20`

Запустіть кожну програму кілька разів з різними даними й перевірте результати вручну або на калькуляторі. У завданні 3 запустіть програму стільки разів, щоб побачити всі три варіанти результату (нічию також).

Додатково (за бажанням): у завданні 3 додайте функції параметр `n` — кількість кубиків, щоб `roll(2)` повертала суму двох кубиків.

Де писати код і як поділитися результатом

Усі завдання виконуємо в OnlineGDB. Модулі `math` і `random` входять у стандартну бібліотеку Python, тому працюють одразу після `import`, без жодного встановлення.

Робота в OnlineGDB

1. Відкрийте onlinegdb.com/online_python_compiler. Праворуч угорі має бути вибрана мова **Python 3**.
2. Напишіть програму у файлі `main.py` і натисніть **Run**. Дані для `input()` вводьте в консолі внизу й натискайте Enter.
3. Для завдання 4 натисніть значок **New File** над редактором (поруч із `main.py`) і назвіть файл `geometry.py`. Запускається завжди `main.py`, а `geometry.py` підключається з нього командою `import geometry`.
4. Коли програма працює правильно, натисніть **Share** і скопіюйте посилання.
5. Відкрийте посилання в новій вкладці й перевірте, що там ваш остаточний код (для завдання 4 — обидва файли).
6. Надішліть посилання вчителю в Google Classroom.

7. Для кожного завдання створюйте окреме посилання. Посилання зберігає код на момент натискання Share, тому після будь-яких змін натисніть Share ще раз.

Щоб не втратити код, не закривайте вкладку, поки не отримали посилання: без реєстрації OnlineGDB не зберігає проєкти автоматично.

Запасні варіанти

Якщо OnlineGDB не відкривається, скористайтеся іншим сервісом.

Сервіс	Як отримати посилання	Кілька файлів (завдання 4)	Особливості
online-python.com	**Share** (або F9 чи Ctrl+H) → у полі «Link expires after» виберіть «Never» → «Create link»	так, додаткові вкладки-файли	F10 тут не ділиться кодом, а завантажує файл. Вікно встановлення пакетів потрібне лише для сторонніх бібліотек на кшталт `numpy`. Якщо під час набору зникають літери, перевірте в налаштуваннях, що «Key bindings» = Default
Programiz	**Share → Copy Link**	ні, лише `main.py`	найпростіший інтерфейс, підходить для завдань 1–3
Python Tutor	**Visualize Execution → Generate permanent link**	ні	покроково показує, як викликається функція й куди повертається `return`; зручно, щоб розібратися з помилкою